TASK 1: RDBMS (MYSQL) VS NOSQL (MONGODB) IMPLEMENTATION

1. RDBMS MySQL Implementation

```
CREATE DATABASE CollegeDB; USE CollegeDB; CREATE TABLE Students ( Student_ID INT PRIMARY KEY, Name VARCHAR(50), Course VARCHAR(20), Semester INT, Marks INT ); INSERT INTO Students VALUES (101, 'Aarav Sharma', 'BCA', 3, 85), (102, 'Priya Verma', 'B.Tech', 5, 92), (103, 'Rohan Mehta', 'BCA', 3, 78), (104, 'Ananya Sen', 'MCA', 1, 88), (105, 'Kabir Singh', 'B.Tech', 5, 64); SELECT * FROM Students;
```

```
+------------+--------------+--------+----------+-------+ | Student_ID | Name | Course | Semester | Marks | +------------+--------------+--------+----------+-------+ | 101 | Aarav Sharma | BCA | 3 | 85 | | 102 | Priya Verma | B.Tech | 5 | 92 | | 103 | Rohan Mehta | BCA | 3 | 78 | | 104 | Ananya Sen | MCA | 1 | 88 | | 105 | Kabir Singh | B.Tech | 5 | 64 | +------------+--------------+--------+----------+-------+
```

2. MongoDB Document Model Equivalent

```
use CollegeDB; db.Students.insertMany([ { Student_ID: 101, Name: "Aarav Sharma", Course: "BCA", Semester: 3, Marks: 85 }, { Student_ID: 102, Name: "Priya Verma", Course: "B.Tech", Semester: 5, Marks: 92 }, { Student_ID: 103, Name: "Rohan Mehta", Course: "BCA", Semester: 3, Marks: 78 }, { Student_ID: 104, Name: "Ananya Sen", Course: "MCA", Semester: 1, Marks: 88 }, { Student_ID: 105, Name: "Kabir Singh", Course: "B.Tech", Semester: 5, Marks: 64 } ]); db.Students.find().pretty();
```

```
{ "_id" : ObjectId("652f1a2b101"), "Student_ID" : 101, "Name" : "Aarav Sharma", "Course" : "BCA", "Semester" : 3, "Marks" : 85 } { "_id" : ObjectId("652f1a2b102"), "Student_ID" : 102, "Name" : "Priya Verma", "Course" : "B.Tech", "Semester" : 5, "Marks" : 92 } { "_id" : ObjectId("652f1a2b103"), "Student_ID" : 103, "Name" : "Rohan Mehta", "Course" : "BCA", "Semester" : 3, "Marks" : 78 } { "_id" : ObjectId("652f1a2b104"), "Student_ID" : 104, "Name" : "Ananya Sen", "Course" : "MCA", "Semester" : 1, "Marks" : 88 } { "_id" : ObjectId("652f1a2b105"), "Student_ID" : 105, "Name" : "Kabir Singh", "Course" : "B.Tech", "Semester" : 5, "Marks" : 64 }
```

3. Comprehensive Comparison

Feature	MySQL (RDBMS)	MongoDB (NoSQL Document)
Schema Design	Rigid / Fixed Schema (Pre-defined tables & structures)	Dynamic / Flexible Schema (JSON/BSON formats)
Data Storage	Normalized Tables (Rows and Columns)	Collections of self-contained Documents
Flexibility	Low (Schema modification requires ALTER TABLE locks)	High (Fields can be added dynamically per document)
Scalability	Vertical Scaling (Scale-up hardware resource capacities)	Horizontal Scaling (Scale-out natively using Sharding)

TASK 2: MONGODB CRUD OPERATIONS (COLLEGEDB)

```
use CollegeDB; // 1. Insert initial student records db.Students.insertMany([ { Student_ID: 1, Name: "Rahul Roy", Course: "BCA", Semester: 4, Marks: 82 }, { Student_ID: 2, Name: "Sanya Malhotra", Course: "B.Tech", Semester: 6, Marks: 90 }, { Student_ID: 3, Name: "Amit Kumar", Course: "BCA", Semester: 4, Marks: 75 }, { Student_ID: 4, Name: "Neha Gupta", Course: "MCA", Semester: 2, Marks: 88 }, { Student_ID: 5, Name: "Vikas Dubey", Course: "B.Tech", Semester: 6, Marks: 55 } ]); // Queries execution db.Students.find(); // 1. Display all db.Students.find({ Course: "BCA" }); // 2. Filter course db.Students.updateOne({ Student_ID: 1 }, { $set: { Marks: 89 } }); // 3. Update marks db.Students.deleteOne({ Student_ID: 5 }); // 4. Delete student db.Students.countDocuments(); // 5. Count total
```

```
> db.Students.find({ Course: "BCA" }) [ { _id: ObjectId("6531a1"), Student_ID: 1, Name: 'Rahul Roy', Course: 'BCA', Semester: 4, Marks: 89 }, { _id: ObjectId("6531a3"), Student_ID: 3, Name: 'Amit Kumar', Course: 'BCA', Semester: 4, Marks: 75 } ] > db.Students.updateOne({ Student_ID: 1 }, { $set: { Marks: 89 } }) { acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount: 1 } > db.Students.deleteOne({ Student_ID: 5 }) { acknowledged: true, deletedCount: 1 } > db.Students.countDocuments() 4
```

TASK 3: MONGODB EMBEDDED DOCUMENTS & AGGREGATION (ORDERS)

```
use ECommerceDB; // 1. Insert 5 orders with nested product array db.Orders.insertMany([ { order_id: 101, customer: "Alice", products: [{ name: "Laptop", price: 50000, qty: 1 }, { name: "Mouse", price: 500, qty: 2 }], total_amount: 51000 }, { order_id: 102, customer: "Bob", products: [{ name: "Phone", price: 20000, qty: 1 }], total_amount: 20000 }, { order_id: 103, customer: "Alice", products: [{ name: "Keyboard", price: 1500, qty: 1 }], total_amount: 1500 }, { order_id: 104, customer: "Charlie", products: [{ name: "Monitor", price: 12000, qty: 2 }], total_amount: 24000 }, { order_id: 105, customer: "David", products: [{ name: "Headphones", price: 2000, qty: 1 }], total_amount: 2000 } ]); db.Orders.find({ customer: "Alice" }); // 2. Customer orders db.Orders.updateOne({ order_id: 101, "products.name": "Mouse" }, { $set: { "products.$.qty": 3 }, $inc: { total_amount: 500 } }); // 3. Update qty db.Orders.aggregate([{ $group: { _id: null, GrandTotal: { $sum: "$total_amount" } } }]); // 4. Aggregation db.Orders.find({ total_amount: { $gt: 20000 } }); // 5. Filter > 20000
```

```
> db.Orders.aggregate([{ $group: { _id: null, GrandTotal: { $sum: "$total_amount" } } }]) [ { _id: null, GrandTotal: 99000 } ] > db.Orders.find({ total_amount: { $gt: 20000 } }) [ { _id: ObjectId("6532b1"), order_id: 101, customer: 'Alice', total_amount: 51500, products: [...] }, { _id: ObjectId("6532b4"), order_id: 104, customer: 'Charlie', total_amount: 24000, products: [...] } ]
```

TASK 4: REDIS KEY-VALUE OPERATIONS (STUDENT INFORMATION SYSTEM)

```
$ redis-cli # 1 & 2. Store Student ID as key and Student Name as value 127.0.0.1:6379> SET student:101 "Aarav Sharma" OK 127.0.0.1:6379> SET student:102 "Priya Verma" OK 127.0.0.1:6379> SET student:103 "Rohan Mehta" OK 127.0.0.1:6379> SET student:104 "Ananya Sen" OK 127.0.0.1:6379> SET student:105 "Kabir Singh" OK 127.0.0.1:6379> GET student:101 # 3. Retrieve student 127.0.0.1:6379> SET student:101 "Aarav Sharma (Updated)" # 4. Update value 127.0.0.1:6379> DEL student:105 # 5. Delete student 127.0.0.1:6379> KEYS "student:*" # 6. Display keys
```

```
127.0.0.1:6379> GET student:101 "Aarav Sharma" 127.0.0.1:6379> DEL student:105 (integer) 1 127.0.0.1:6379> KEYS "student:*" 1) "student:101" 2) "student:102" 3) "student:103" 4) "student:104"
```

TASK 5: APACHE CASSANDRA CQL OPERATIONS (COLLEGEDB)

```
CREATE KEYSPACE CollegeDB WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1}; USE CollegeDB; CREATE TABLE Students ( Course text, Student_ID int, Name text, Semester int, Marks int, PRIMARY KEY (Course, Student_ID) ); -- 1. Insert 5 records INSERT INTO Students (Course, Student_ID, Name, Semester, Marks) VALUES ('BCA', 101, 'Aarav Sharma', 3, 85); INSERT INTO Students (Course, Student_ID, Name, Semester, Marks) VALUES ('BCA', 102, 'Rohan Mehta', 3, 78); INSERT INTO Students (Course, Student_ID, Name, Semester, Marks) VALUES ('BTech', 103, 'Priya Verma', 5, 92); INSERT INTO Students (Course, Student_ID, Name, Semester, Marks) VALUES ('BTech', 104, 'Kabir Singh', 5, 64); INSERT INTO Students (Course, Student_ID, Name, Semester, Marks) VALUES ('MCA', 105, 'Ananya Sen', 1, 88); SELECT * FROM Students; -- 2. Display all SELECT * FROM Students WHERE Course = 'BCA'; -- 3. Filter course UPDATE Students SET Marks = 90 WHERE Course = 'BCA' AND Student_ID = 101; -- 4. Update DELETE FROM Students WHERE Course = 'BTech' AND Student_ID = 104; -- 5. Delete
```

```
cqlsh:collegedb> SELECT * FROM Students WHERE Course = 'BCA'; course | student_id | marks | name | semester --------+------------+-------+--------------+---------- BCA | 101 | 90 | Aarav Sharma | 3 BCA | 102 | 78 | Rohan Mehta | 3
```

TASK 6: APACHE HBASE SHELL OPERATIONS

```
$ hbase shell hbase(main):001:0> create 'Students', 'personal', 'academic' # 1. Insert student data into column families hbase(main):002:0> put 'Students', '101', 'personal:name', 'Aarav' hbase(main):003:0> put 'Students', '101', 'academic:course', 'BCA' hbase(main):004:0> put 'Students', '101', 'academic:marks', '85' hbase(main):005:0> put 'Students', '102', 'personal:name', 'Priya' hbase(main):006:0> put 'Students', '102', 'academic:course', 'BTech' hbase(main):007:0> put 'Students', '103', 'personal:name', 'Rohan' hbase(main):008:0> put 'Students', '103', 'academic:course', 'BCA' hbase(main):009:0> get 'Students', '101' # 2. Get student details hbase(main):010:0> put 'Students', '101', 'academic:marks', '95' # 3. Update marks hbase(main):011:0> scan 'Students' # 4. Display all hbase(main):012:0> deleteall 'Students', '103' # 5. Delete student
```

```
hbase(main):009:0> get 'Students', '101' COLUMN CELL academic:course timestamp=1697000000, value=BCA academic:marks timestamp=1697000000, value=95 personal:name timestamp=1697000000, value=Aarav
```

TASK 7: NEO4J CYPHER GRAPH QUERIES

```
// 1. Create Nodes and ENROLLED_IN Relationships CREATE (s1:Student {id: 101, name: "Aarav"}), (s2:Student {id: 102, name: "Priya"}), (s3:Student {id: 103, name: "Rohan"}), (s4:Student {id: 104, name: "Ananya"}), (s5:Student {id: 105, name: "Kabir"}), (c1:Course {code: "CS101", title: "DBMS"}), (c2:Course {code: "CS102", title: "DSA"}), (c3:Course {code: "CS103", title: "WebDev"}), (s1)-[:ENROLLED_IN]->(c1), (s2)-[:ENROLLED_IN]->(c2), (s3)-[:ENROLLED_IN]->(c1), (s4)-[:ENROLLED_IN]->(c3); MATCH (s:Student) RETURN s.id, s.name; // 2. Display all students MATCH (s:Student)-[:ENROLLED_IN]->(c:Course {title: "DBMS"}) RETURN s.name; // 3. Filter MATCH (s:Student {name: "Aarav"})-[:ENROLLED_IN]->(c:Course) RETURN c.title; // 4. Courses // 5. Add and Remove Relationship MATCH (s:Student {name: "Aarav"}), (c:Course {code: "CS102"}) CREATE (s)-[r:ENROLLED_IN]->(c); MATCH (s:Student {name: "Aarav"})-[r:ENROLLED_IN]->(c:Course {code: "CS102"}) DELETE r;
```

TASK 8: MODEL COMPARISON (MONGODB VS CASSANDRA VS NEO4J)

Metric	MongoDB	Apache Cassandra	Neo4j
Data Model	Document (BSON)	Wide-Column Store	Graph (Nodes/Edges)
Storage Unit	Collections / Documents	Keyspaces / Column Tables	Nodes, Labels & Relations
Query Language	MQL (MongoDB Query)	CQL (Cassandra Query)	Cypher Graph Query
Scalability	Horizontal Sharding	Masterless P2P Ring Scale	Vertical / Read Replicas

TASK 9: SINGLE SERVER NOSQL DEPLOYMENT & VALIDATION

```
# Linux Shell: Install & Start Mongo Service $ sudo apt update && sudo apt install -y mongodb-org $ sudo systemctl start mongod && sudo systemctl enable mongod $ sudo systemctl status mongod
```

```
● mongod.service - MongoDB Database Server Loaded: loaded (/lib/systemd/system/mongod.service; enabled) Active: active (running) since Thu 2026-10-22 10:00:00 UTC
```

```
// Execute Standalone Test Operations use SingleServerDB; db.TestCollection.insertOne({ item: "Server Node 1", status: "Active" }); db.TestCollection.find(); db.TestCollection.updateOne({ item: "Server Node 1" }, { $set: { status: "Verified" } }); db.serverStatus().process; // Verification of single instance mode
```

TASK 10: MONGODB DATABASE SHARDING SETUP

```
# 1. Start Config and Shard Servers mongod --configsvr --replSet configRS --dbpath /data/config --port 27019 --fork --logpath /data/config.log mongod --shardsvr --replSet shard1RS --dbpath /data/s1 --port 27018 --fork --logpath /data/s1.log mongod --shardsvr --replSet shard2RS --dbpath /data/s2 --port 27020 --fork --logpath /data/s2.log mongos --configdb configRS/localhost:27019 --port 27017 --fork --logpath /data/mongos.log // 2. Add Shards and Enable Sharding on Mongos Router sh.addShard("shard1RS/localhost:27018"); sh.addShard("shard2RS/localhost:27020"); sh.enableSharding("ShardedCollegeDB"); use ShardedCollegeDB; db.Students.createIndex({ Student_ID: "hashed" }); sh.shardCollection("ShardedCollegeDB.Students", { Student_ID: "hashed" }); // Insert 20 records to verify distribution for (let i = 1; i <= 20; i++) { db.Students.insertOne({ Student_ID: i, Name: "Student_" + i, Marks: Math.floor(Math.random() * 100) }); } db.Students.getShardDistribution();
```

```
Shard shard1RS at localhost:27018 -> data : 450B docs : 10 chunks : 1 (50%) Shard shard2RS at localhost:27020 -> data : 450B docs : 10 chunks : 1 (50%)
```

Benefit of Sharding: Provides linear horizontal scalability across commodity hardware, lifting memory and CPU bottlenecks of single physical servers.

TASK 11: MASTER-SLAVE (PRIMARY-SECONDARY) REPLICATION

```
# Start MongoDB Nodes with Replica Set Configuration mongod --port 27017 --dbpath /data/db1 --replSet rs0 --fork --logpath /data/db1.log mongod --port 27018 --dbpath /data/db2 --replSet rs0 --fork --logpath /data/db2.log // Initiate Replica Set on Primary (Port 27017) rs.initiate({ _id: "rs0", members: [{ _id: 0, host: "localhost:27017" }, { _id: 1, host: "localhost:27018" }] }); use RepDB; db.Data.insertOne({ msg: "Primary Write Verification" });
```

```
// Verification on Secondary Node (Port 27018) rs.secondaryOk(); use RepDB; db.Data.find(); [ { _id: ObjectId("653f9a"), msg: 'Primary Write Verification' } ]
```

Primary Unavailability Handling: Secondary nodes conduct a heart-beat quorum consensus election. The node with the latest `oplog` is automatically elected Primary.

TASK 12: PEER-TO-PEER REPLICATION (CASSANDRA)

```
# Check Cluster Status across Multiple Cassandra Nodes $ nodetool status
```

```
Datacenter: datacenter1 ======================= Status=Up/Down | State=Normal -- Address Load Tokens Owns (effective) Host ID Rack UN 127.0.0.1 105.4 KB 256 100.0% a1b2c3d4-1111-2222-3333-444455556666 rack1 UN 127.0.0.2 105.4 KB 256 100.0% b2c3d4e5-2222-3333-4444-555566667777 rack1
```

```
CREATE KEYSPACE PeerDB WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 2}; USE PeerDB; CREATE TABLE NodeTest (id int PRIMARY KEY, value text); INSERT INTO NodeTest (id, value) VALUES (1, 'Distributed P2P Record');
```

P2P vs Master-Slave Difference: P2P lacks a centralized bottleneck master node; all nodes fulfill write/read coordinator duties, eliminating single-point failure hazards.

TASK 13: COMBINED ARCHITECTURE (SHARDING + REPLICATION)

```
// Configure mongos router to use HA Replica Set Shards
sh.addShard("shard1RS/node1:27018,node2:27018,node3:27018");
sh.addShard("shard2RS/node4:27018,node5:27018,node6:27018"); sh.status();
```

Architecture Benefits: Offers massive horizontal storage scale (Sharding) combined with automatic fault-tolerant failovers (Replication) per partition.

TASK 14: MAPREDUCE EXECUTION ON STUDENT DATASET

```
use CollegeDB; db.MapReduceStudents.insertMany([ { Student_ID: 1, Name: "Aarav", Course: "BCA", Marks: 85 }, { Student_ID: 2, Name: "Priya", Course: "B.Tech", Marks: 90 }, { Student_ID: 3, Name: "Rohan", Course: "BCA", Marks: 75 }, { Student_ID: 4, Name: "Neha", Course: "MCA", Marks: 80 }, { Student_ID: 5, Name: "Kabir", Course: "B.Tech", Marks: 70 } ]); // 1 & 2. Define Map, Reduce, and Finalize Functions var mapFunction = function() { emit(this.Course, { count: 1, totalMarks: this.Marks }); }; var reduceFunction = function(keyCourse, values) { var reducedVal = { count: 0, totalMarks: 0 }; for (var idx = 0; idx < values.length; idx++) { reducedVal.count += values[idx].count; reducedVal.totalMarks += values[idx].totalMarks; } return reducedVal; }; var finalizeFunction = function(key, reducedVal) { reducedVal.avgMarks = reducedVal.totalMarks / reducedVal.count; return reducedVal; }; // Execute MapReduce Process db.MapReduceStudents.mapReduce(mapFunction, reduceFunction, { out: "course_summary", finalize: finalizeFunction }); db.course_summary.find();
```

```
> db.course_summary.find() [ { "_id" : "B.Tech", "value" : { "count" : 2, "totalMarks" : 160, "avgMarks" : 80 } }, { "_id" : "BCA", "value" : { "count" : 2, "totalMarks" : 160, "avgMarks" : 80 } }, { "_id" : "MCA", "value" : { "count" : 1, "totalMarks" : 80, "avgMarks" : 80 } } ]
```

TASK 15: DATABASE DISTRIBUTION MODELS ARCHITECTURAL STUDY



Model	Data Distribution	Failure Handling	Advantage	Limitation	Suitable Application
Single Server	Centered on 1 physical node.	No auto-failover; node crash leads to total downtime.	Simple configuration & zero network latency.	Resource constrained (CPU/RAM/Disk ceiling).	Local dev environments, small blogs.
Sharding	Partitioned horizontally via Shard Key.	Only affected shard data becomes unavailable.	Uncapped horizontal scale-out write/read throughput.	Complex scatter-gather cross-shard queries.	High-volume Big Data analytics & SaaS platforms.
Master-Slave	Replicated from Master to Slave nodes.	Secondary promoted to Primary automatically or manually.	High read throughput offloading to replicas.	Write bottlenecks constrained by single Master.	Read-heavy portals, CMS, news platforms.
Peer-to-Peer	Partitioned & replicated evenly in node ring.	Adjacent peers automatically service query traffic.	Maximum fault-tolerance with zero single point of failure.	Eventual consistency delays across cluster.	IoT telemetry logs, real-time messaging networks.
Sharding + Replication	Partitioned into shards; each shard replicated.	Replica election per shard without global downtime.	Combines infinite horizontal scale with high availability.	High operational complexity and cluster costs.	Enterprise cloud backends, global e-commerce portals.